ASE CHAPTER 4

Multiple Choice Questions · SW Project Planning

Q4.1. After the finalization of SRS, we may like to estimate:

(a) Size · (b) Cost · (c) Development time · (d) All of the above

(d) All of the above

Software project planning begins once the SRS is baseline-approved; the primary estimation objectives are software size, project cost/effort, development schedule/time, and required staffing. These estimates are related: size is an input to effort models, and effort and schedule estimates contribute to project planning and cost.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 139; Companion Slide 194. [Textbook Answer Key]

Q4.2. Which one is not a size measure for software?

(a) LOC · (b) Function Count · (c) Cyclomatic Complexity · (d) Halstead's program length

(c) Cyclomatic Complexity

Lines of Code (LOC), Function Count / Function-Point measures, and Halstead's Program Length ($N = N_1 + N_2$) are software-size measures. Function Count and the resulting Function Points are related steps, not interchangeable labels. Cyclomatic Complexity ($V(G) = e - n + 2P$), introduced by Thomas McCabe, is a structural complexity metric measuring the number of linearly independent paths through the control flow graph, not software size.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 143–150; Companion Slide 194. [Textbook Answer Key]

Q4.3. Function count method was developed by:

(a) B. Beizer · (b) B. Boehm · (c) M. Halstead · (d) Alan Albrecht

(d) Alan Albrecht

The Function Count / Function Point Analysis (FPA) method was originally developed by Allan J. Albrecht at IBM in 1979 to measure the functional size of software from the user's perspective, independent of programming language or implementation technology.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 145; Companion Slide 194. [Textbook Answer Key]

Q4.4. Function point analysis (FPA) method decomposes the system into functional units. The total number of functional units are:

(a) 2 · (b) 5 · (c) 4 · (d) 1

(b) 5

Function Point Analysis decomposes a software system into five distinct functional information domain characteristics: External Inputs (EI), External Outputs (EO), External Inquiries (EQ), Internal Logical Files (ILF), and External Interface Files (EIF).

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 145–146; Companion Slide 194. [Textbook Answer Key]

Q4.5. IFPUG stands for:

(a) Initial function point uniform group · (b) International function point uniform group · (c) International function point user group · (d) Initial function point user group

> **(c) International function point user group**
>
> **IFPUG stands for the International Function Point Users Group.**
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 145; Companion Slide 195. [Textbook Answer Key]*

Q4.6. Function points can be calculated by:

(a) UFP * CAF · (b) UFP * FAC · (c) UFP * Cost · (d) UFP * Productivity

> **(a) UFP * CAF**
>
> **Final adjusted Function Points (FP) are calculated by multiplying Unadjusted Function Points (UFP) by the Complexity Adjustment Factor (CAF):** $\mathrm{FP} = \mathrm{UFP} \times \mathrm{CAF}$.
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 147; Companion Slide 195. [Textbook Answer Key]*

Q4.7. Putnam resource allocation model is based on:

(a) Function points · (b) Norden/Rayleigh curve · (c) Putnam theory of software management · (d) Boehm's observation on manpower utilisation rate

> **(b) Norden/Rayleigh curve**
>
> **The Putnam Resource Allocation Model is founded directly upon Peter Norden's empirical observations at IBM, which established that manpower utilization throughout research and development projects follows a Rayleigh distribution curve over time.**
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 181–182; Companion Slide 195. [Textbook Answer Key]*

Q4.8. Manpower buildup for Putnam resource allocation model is:

(a) $D = K/t_d$ · (b) $D_0 = K/t_d^3$ · (c) $D = K/t_d^2$ · (d) $D_0 = K/t_d^2$

> **(b)** $D_0 = K/t_d^3$
>
> **In the Putnam resource allocation model, the difficulty metric is** $D = K/t_d^2$**, while the separate manpower build-up metric is** $D_0 = K/t_d^3$ **(persons/year^2).**
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 186; Companion Slide 195. [Textbook Answer Key]*

Q4.9. COCOMO was developed initially by:

(a) B. W. Boehm · (b) Gregg Rothermel · (c) B. Beizer · (d) Rajiv Gupta

> **(a) B. W. Boehm**
>
> **The Constructive Cost Model (COCOMO '81) was developed by Barry W. Boehm.**
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 152; Companion Slide 196. [Textbook Answer Key]*

Q4.10. A COCOMO model is:

(a) Common Cost estimation model · (b) Constructive Cost Estimation model · (c) Complete cost estimation model · (d) Comprehensive Cost estimation model

> **(b) Constructive Cost Estimation model**
>
> **COCOMO is an acronym for COnstructive COst MOdel, an algorithmic cost estimation model that estimates software effort, schedule, and team size based on project size (KLOC) and cost drivers.**
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 152; Companion Slide 196. [Textbook Answer Key]*

Q4.11. Estimation of software development effort for organic software in COCOMO is:

(a) $E = 2.4(\text{KLOC})^{1.05}\ \text{PM}$ · (b) $E = 3.4(\text{KLOC})^{1.06}\ \text{PM}$ · (c) $E = 2.0(\text{KLOC})^{1.05}\ \text{PM}$ · (d) $E = 2.4(\text{KLOC})^{1.07}\ \text{PM}$

> **(a)** $E = 2.4(\text{KLOC})^{1.05}\ \text{PM}$
>
> **In Basic COCOMO '81, the nominal development effort equation for the Organic mode is** $E = 2.4 \times (\text{KLOC})^{1.05}\ \text{PM}$**.**
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 153; Companion Slide 196. [Textbook Answer Key]*

Q4.12. Estimation of size for a project is dependent on:

(a) Cost · (b) Schedule · (c) Time · (d) None of the above

> **(d) None of the above**
>
> **Software size is an intrinsic attribute of the system's requirements and functionality; it is independent of cost, schedule, or calendar time.** In fact, cost and schedule are dependent variables estimated *from* the estimated size, not the other way around.
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 143; Companion Slide 196. [Textbook Answer Key]*

Q4.13. In function point analysis, number of Complexity Adjustment Factors are:

(a) 10 · (b) 20 · (c) 14 · (d) 12

> **(c) 14**
>
> **Function Point Analysis defines exactly 14 General System Characteristics (GSCs) as complexity adjustment factors (such as data communications, distributed data processing, performance, transaction rate, online data entry, etc.), each rated from 0 to 5.**
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 146–147; Companion Slide 196. [Textbook Answer Key]*

Q4.14. COCOMO-II estimation model is based on:

(a) Complex approach · (b) Algorithmic approach · (c) Bottom-up approach · (d) Top-down approach

(b) Algorithmic approach

COCOMO II is an algorithmic cost estimation model that uses mathematical formulas derived from empirical regression analysis of historical project metrics to predict effort and schedule.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 161; Companion Slide 197. [Textbook Answer Key]

Q4.15. Cost estimation for a project may include:

(a) Software Cost · (b) Hardware Cost · (c) Personnel Costs · (d) All of the above

(d) All of the above

The answer includes software cost, hardware cost, and personnel costs; these are all relevant components of a project cost estimate.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 150; Companion Slide 197. [Textbook Answer Key]

Q4.16. In COCOMO model, if project size is typically 2–50 KLOC, then which mode is to be selected?

(a) Organic · (b) Semidetached · (c) Embedded · (d) None of the above

(a) Organic

In COCOMO, the Organic mode represents relatively small software projects (typically 2 to 50 KLOC) developed by experienced teams working in familiar, stable in-house environments with relaxed requirements.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 152; Companion Slide 197. [Textbook Answer Key]

Q4.17. COCOMO-II was developed at:

(a) University of Maryland · (b) University of Southern California · (c) IBM · (d) AT&T Bell Labs

(b) University of Southern California

COCOMO II was developed by Barry Boehm and his research team at the Center for Software Engineering (CSE) at the University of Southern California (USC) during the 1990s.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 161; Companion Slide 197. [Textbook Answer Key]

Q4.18. Which one is not a Category of COCOMO-II?

(a) End User Programming · (b) Infrastructure Sector · (c) Requirement Sector · (d) System Integration

(c) Requirement Sector

COCOMO II identifies four target project categories / application sectors: End-User Programming, Application Generators & Composition Aids, System Integration, and Infrastructure Sector.
"Requirement Sector" does not exist in COCOMO II taxonomy.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 162; Companion Slide 197. [Textbook Answer Key]

Q4.19. Which one is not an infrastructure software?

(a) Operating system · (b) Database management system · (c) Compilers · (d) Result management system

(d) Result management system

Infrastructure software provides core foundational services and platforms upon which application software runs; examples include Operating Systems, DBMS, Compilers, Network protocols, and Device Drivers. "Result management system" is an end-user application, not infrastructure software.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 162; Companion Slide 198. [Textbook Answer Key]

Q4.20. How many stages are in COCOMO-II?

(a) 2 · (b) 3 · (c) 4 · (d) 5

(b) 3

COCOMO II provides three increasingly detailed estimation stages tailored to the life cycle: Stage 1 (Application Composition Model), Stage 2 (Early Design Model), and Stage 3 (Post-Architecture Model).

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 163; Companion Slide 198. [Textbook Answer Key]

Q4.21. Which one is not a stage of COCOMO-II?

(a) Application Composition estimation model · (b) Early design estimation model · (c) Post architecture estimation model · (d) Comprehensive cost estimation model

(d) Comprehensive cost estimation model

The three official stages of COCOMO II are Application Composition, Early Design, and Post-Architecture. "Comprehensive cost estimation model" is an invented distractor.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 163; Companion Slide 198. [Textbook Answer Key]

Q4.22. In Putnam resource allocation model, Rayleigh curve is modeled by the equation:

(a) $m(t) = 2at \cdot e^{-at^2}$ · (b) $m(t) = 2Kt \cdot e^{-at^2}$ · (c) $m(t) = 2Kat \cdot e^{-at^2}$ · (d) $m(t) = 2Kbt \cdot e^{-at^2}$

(c) $m(t) = 2Kat \cdot e^{-at^2}$

In the Putnam resource allocation model, manpower utilization rate $m(t)$ over time follows Norden's Rayleigh curve equation: $m(t) = 2Kat \cdot e^{-at^2}$, where K is total life-cycle effort, $a = \frac{1}{2t_d^2}$, and t is elapsed project time.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 182–183; Companion Slide 198. [Textbook Answer Key]

Q4.23. In Putnam resource allocation model, technology factor 'C' is defined as:

(a) $C = S \cdot K^{-1/3} \cdot t_d^{-4/3}$ · (b) $C = S \cdot K^{1/3} \cdot t_d^{4/3}$ · (c) $C = S \cdot K^{-1/3} \cdot t_d^{4/3}$ · (d) $C = S \cdot K^{1/3} \cdot t_d^{-4/3}$

(a) $C = S \cdot K^{-1/3} \cdot t_d^{-4/3}$

Putnam's Software Equation relates product size S to total effort K and development time t_d by $S = C \cdot K^{1/3} \cdot t_d^{4/3}$. Solving algebraically for the technology factor C yields $C = S \cdot K^{-1/3} \cdot t_d^{-4/3}$.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 187; Companion Slide 199. [Textbook Answer Key]

Q4.24. Risk management activities are divided in:

(a) 3 Categories · (b) 2 Categories · (c) 5 Categories · (d) 10 Categories

(b) 2 Categories

Risk management activities are divided into two fundamental primary categories: Risk Assessment (identification, analysis, prioritization) and Risk Control (planning, resolution, monitoring).

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 200; Companion Slide 199. [Textbook Answer Key]

Q4.25. Which one is not a risk management activity?

(a) Risk assessment · (b) Risk control · (c) Risk generation · (d) None of the above

(c) Risk generation

The established activities of software risk management are Risk Assessment (evaluating exposure) and Risk Control (mitigating and monitoring risk). "Risk generation" is not one of the two listed management activities.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 200; Companion Slide 199. [Textbook Answer Key]

Q4.26. In the Putnam Resource Allocation Model, the peak manpower manning rate m_0 at time $t = t_d$ is mathematically given by:

(a) $m_0 = \frac{K}{t_d \cdot e}$ · (b) $m_0 = \frac{K}{t_d \sqrt{e}}$ · (c) $m_0 = \frac{2K}{t_d \sqrt{e}}$ · (d) $m_0 = \frac{K}{t_d^2 \sqrt{e}}$

(b) $m_0 = \frac{K}{t_d \sqrt{e}}$

Substituting $a = \frac{1}{2t_d^2}$ and $t = t_d$ into Norden's Rayleigh curve $m(t) = 2Kat \cdot e^{-at^2}$ yields $m_0 = 2K\left(\frac{1}{2t_d^2}\right) t_d \cdot e^{-\frac{t_d^2}{2t_d^2}} = \frac{K}{t_d} \cdot e^{-1/2} = \frac{K}{t_d \sqrt{e}}$. The expression with e alone in the denominator does not follow from this substitution; with $\sqrt{e} \approx 1.64872$, $m_0 \approx 0.6065 \frac{K}{t_d}$.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 184; Companion Slides 130 & 139. [Course Source Invariant]

Q4.27. In Putnam's Software Equation ($S = C_k \cdot K^{1/3} \cdot t_d^{4/3}$), compressing development time t_d by a factor of 2 while keeping product size S and technology C_k constant will cause required effort K to increase by a factor of:

(a) 2 · (b) 4 · (c) 8 · (d) 16

> **(d) 16**
>
> **Rearranging Putnam's software equation gives $K = \frac{S^3}{C_k^3 \cdot t_d^4}$, demonstrating that development effort K is inversely proportional to the fourth power of development time ($K \propto t_d^{-4}$).** If t_d is halved ($t_d/2$), effort increases by $(1/2)^{-4} = 2^4 = 16$ times.
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 187–188; Companion Slide 166. [Course Source Invariant]*

Q4.28. In Intermediate COCOMO '81, how many total cost drivers (Effort Multipliers) are defined, and across how many categories are they grouped?

(a) 15 drivers across 4 categories · (b) 17 drivers across 4 categories · (c) 14 drivers across 3 categories · (d) 7 drivers across 2 categories

> **(a) 15 drivers across 4 categories**
>
> **Intermediate COCOMO defines exactly 15 cost drivers (effort multipliers) clustered into four primary categories: Product attributes (3: RELY, DATA, CPLX), Computer attributes (4: TIME, STOR, VIRT, TURN), Personnel attributes (5: ACAP, AEXP, PCAP, VEXP, LEXP), and Project attributes (3: MODP, TOOL, SCED).**
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 155–158; Companion Slides 57–61. [Course Source Invariant]*

Q4.29. In COCOMO II, the effort exponent B for the Early Design and Post-Architecture models is computed from 5 scale factors (SF_j) using which formula?

(a) $B = 0.91 + 0.01 \sum_{j=1}^{5} SF_j$ · (b) $B = 1.01 + 0.05 \sum_{j=1}^{5} SF_j$ · (c) $B = 0.91 + 0.1 \sum_{j=1}^{5} SF_j$ · (d) $B = 1.05 + 0.01 \sum_{j=1}^{5} SF_j$

> **(a)** $B = 0.91 + 0.01 \sum_{j=1}^{5} SF_j$
>
> **In COCOMO II, the scale factor exponent B reflects economies or diseconomies of scale and is computed as $B = 0.91 + 0.01 \sum_{j=1}^{5} SF_j$, where the five scale factors are Precedentedness (PREC), Development Flexibility (FLEX), Architecture/Risk Resolution (RESL), Team Cohesion (TEAM), and Process Maturity (PMAT).**
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 169; Companion Slide 86. [Course Source Invariant]*

Q4.30. In Function Point Analysis, which information domain characteristic carries the highest possible individual complexity weight?

(a) External Inputs (EI) · (b) External Outputs (EO) · (c) Internal Logical Files (ILF) · (d) External Interface Files (EIF)

(c) Internal Logical Files (ILF)

In the function point weight matrix, Internal Logical Files (ILF) carry complexity weights of 7 (Low), 10 (Average), and 15 (High), which is the highest weight in the matrix. External Interface Files (EIF) carry 5, 7, 10; External Outputs (EO) carry 4, 5, 7; External Inputs (EI) carry 3, 4, 6.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 146; Companion Slide 16. [Course Source Invariant]

Q4.31. For the full-project Rayleigh curve, approximately what percentage of total effort K has accumulated from $t = 0$ to the release time $t = t_d$?

(a) 15% · (b) 25% · (c) 40% · (d) 65%

(c) 40%

**Integrating the full-project Rayleigh curve from $t = 0$ to $t = t_d$ gives cumulative effort $E(t_d) = K(1 - e^{-1/2}) = 0.3935K \approx 40\%K$. This cumulative value is distinct from the separately defined development-sub-cycle total $K_d = K/6$.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 183; Companion Slide 151. [Course Source Invariant]

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- التكامل على منحنى المشروع من 0 إلى t_d يعطي جهداً تراكمياً يقارب 39.35% من K. هذا المقدار لا يساوي جهد دورة التطوير K_d؛ فهما كميتان مختلفتان ($K = 6K_d$).

Q4.32. Comparing single-variable static cost estimation models for a 100 KLOC project, which model predicts a higher nominal effort: Walston-Felix ($E = 5.2L^{0.91}$) or SEL ($E = 1.4L^{0.93}$)?

(a) Walston-Felix predicts significantly higher effort · (b) SEL predicts significantly higher effort · (c) Both predict identical effort · (d) Neither model can be applied at 100 KLOC

(a) Walston-Felix predicts significantly higher effort

For $L = 100$ KLOC: Walston-Felix yields $E = 5.2 \times (100)^{0.91} \approx 5.2 \times 66.07 \approx 343.6$ PM. SEL yields $E = 1.4 \times (100)^{0.93} \approx 1.4 \times 72.44 \approx 101.4$ PM. For these equations and $L = 100$ KLOC, Walston–Felix predicts more than three times the effort of SEL. This comparison alone does not identify the cause or establish which estimate is more accurate.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 151; Companion Slide 37. [Course Source Invariant]

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- لهذا الحجم تعطي معادلة Felix–Walston جهداً أعلى من معادلة SEL؛ لا تحدد هذه المقارنة وحدها سبب الفرق أو أيهما أدق لمشروع معين.

Q4.33. In quantitative software risk analysis, Risk Exposure (RE) is defined mathematically as:

(a) $\mathrm{RE} = P + L$ · (b) $\mathrm{RE} = P \times L$ · (c) $\mathrm{RE} = L/P$ · (d) $\mathrm{RE} = P/L$

(b) $\mathrm{RE} = P \times L$

Risk Exposure (RE) is the product of the probability of incurring a loss due to a risk (P) and the potential magnitude of that loss (L): $\mathrm{RE} = P \times L$.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 197; Companion Slide 191. [Course Source Invariant]

Q4.34. In COCOMO '81, if a project has high product complexity, strict hardware constraints, and an unfamiliar operational environment requiring rigorous interfaces, which mode best fits these conditions?

(a) Organic · (b) Semidetached · (c) Embedded · (d) Prototyping

(c) Embedded

Embedded mode best fits a project with tight hardware/software constraints and a rigid operational environment. Size ranges are typical guidance; the scenario's constraints also matter.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 152–153; Companion Slide 43. [Course Source Invariant]

Q4.35. What is the fundamental conceptual advantage of Function Points (FP) over Lines of Code (LOC) as a software size metric?

(a) FP can be counted automatically by a compiler · (b) FP is independent of the programming language used · (c) FP requires knowing the full source code syntax · (d) FP measures memory word allocation

(b) FP is independent of the programming language used

Function Points measure functionality delivered to the user from the external functional specification (SRS), making the metric completely programming language independent. In contrast, LOC is heavily dependent on programming language expressiveness, indentation styles, and compiler syntax, penalizing higher-level languages.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 147–148; Companion Slide 31. [Course Source Invariant]